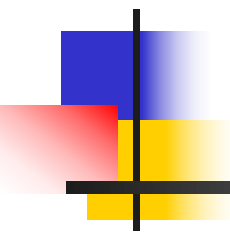


C语言与程序设计

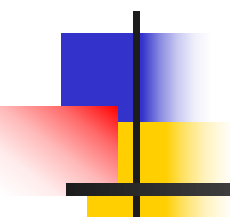
The C Programming Language



第2章 C语言的基本元素

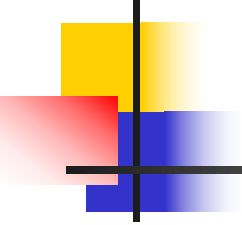
华中科技大学计算机学院

卢萍



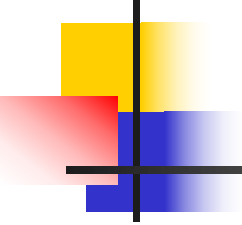
本章内容

- 词法元素
- 标识符和关键字
- 基本数据类型
- 常量与变量
- 运算符和表达式
- 位运算
- 类型转换
- 枚举类型



2.1 字符集及词法元素

- 词法元素称为**记号(token)**
- 记号是程序中**具有语义**的最基本组成单元
- 记号共**分5类**：**标识符、关键字、常量、运算符和标点符号**
- 编译器从左至右收集字符，总是尽量**建立最长的记号**，即使结果并不构成有效的**C语言**程序。相邻记号可以用空白符或注释语句分开。



词法分析举例

- **sum=x+y**

被分解成**sum**、**=**、**x**、**+**和**y** 共5个记号。

- **int a, b=10;**

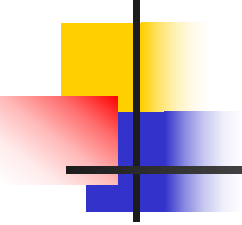
被分解成**int**、**a**、**,**、**b**、**=**、**10**和**;** 共7个记号

- **x+++++y**

被分解成**x**、**++**、**++**、**+**、**y** 共5个记号

2.2 关键字和标识符

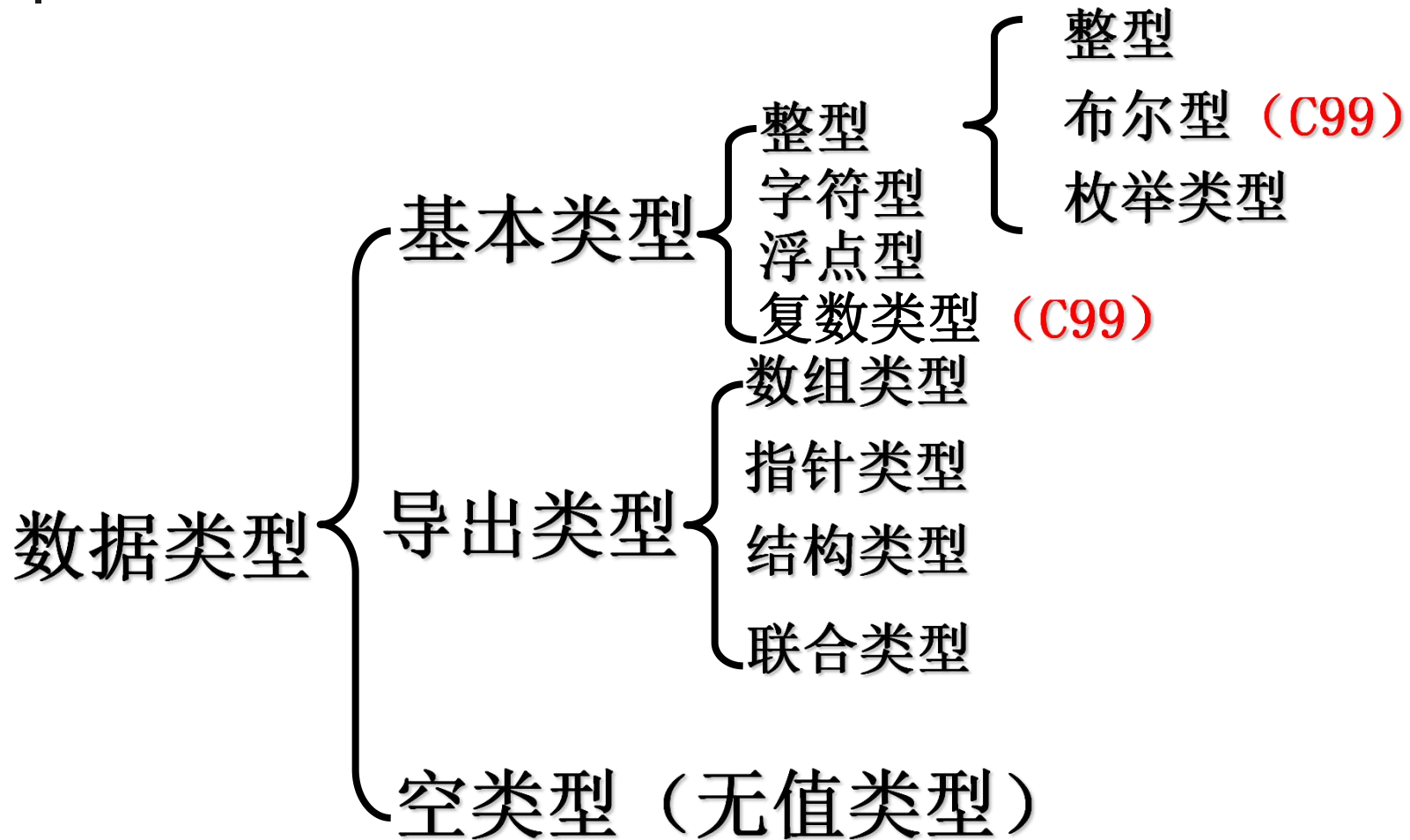
- **关键字**：是被系统赋予特定含义并有专门用途的字，不能作为普通标识符。
 - **int**、**void**、**return**、**while**、**if**
 - 小写
- **标识符**：是用来标识用户定义的常量、变量、数据类型和函数等名字的符号。
 - **命名规则**：以一个字母(**a~z, A~Z**)或下划线(**_**)开头，后跟字母、下划线或数字(**0~9**)
 - **K, _id, month, time1** ✓
 - **20_sum, not#me, void** ✗



注意

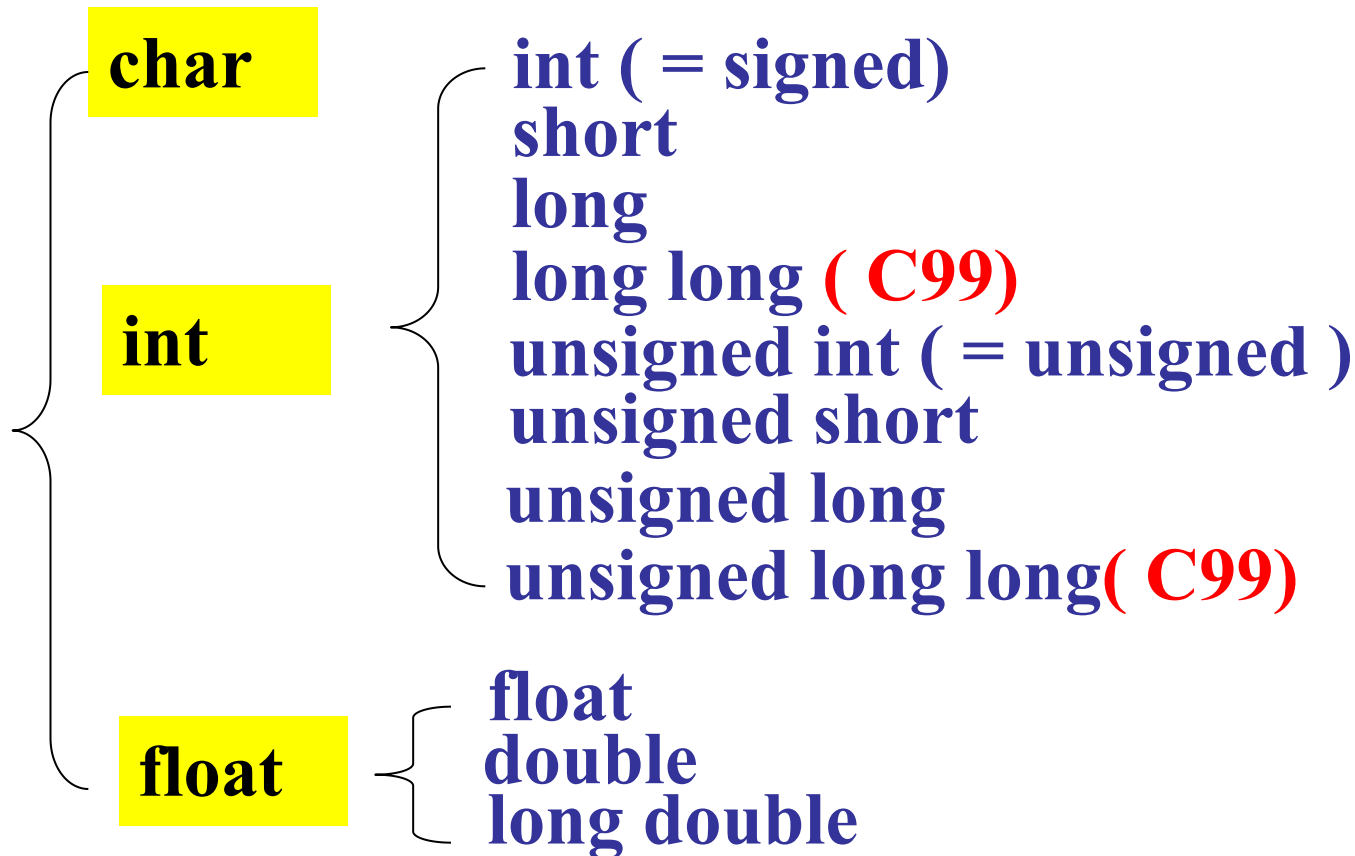
- 大小写字母表示不同意义。
- 不能使用类似 **int** 和 **void** 这样的**C**关键字为自己的对象命名，也要避免使用**C**程序库中函数和常量的名称，例如 **scanf** 。
- 良好的编程风格是选择有助于记忆且**有一定含义的标识符**，这样可增强程序的可读性和程序的文档性。

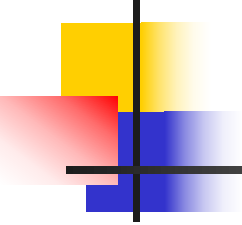
2.3 基本数据类型



基本类型的名字

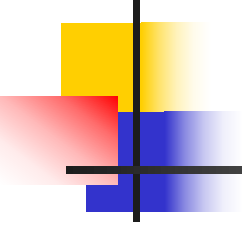
■ 含字符型、整型、浮点型 (表2-2)





char类型

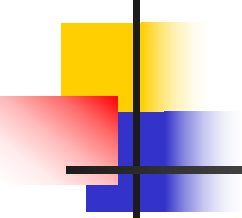
- **char**的存储长度是一字节
- 多数系统中**char**与**signed char**同 (-128~127)
- 字符数据以**ASCII**码存储在内存中
- 在不要求大整数的情况下，可用字符型代替整型。



整型类型

- **int**型值存储在一个机器字中。
- 假设字长为**2B**，**int**取值范围为-32768~32767，**unsigned**取值范围为0 ~ 65535
- 下面的代码是否正确

```
#define BIG 30000
int main(void)
{ short x,y,z;
  x=y=BIG;
  z=x+y;          /* 短整数溢出 */
  .....
}
```

- 
- 程序员必须时刻保证整数表达式的值在合理范围内。
 - 引入**short**和**long**的目的是为了提供各种满足实际要求的不同长度的整数。**int**通常反映特定机器的自然大小，**short**一般为**2B**，**long**一般为**4B**。因此，当关心存储时，用**short**；当需要较大的整数值时，用**long**。

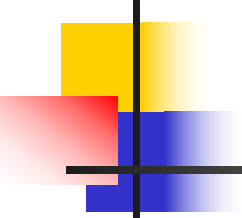
浮点类型

- 一个浮点数N可表示为：
$$V = (-1)^s \times M \times 2^E$$

↓ ↓ ↓
数符 尾数 指数
1b 23b 8b



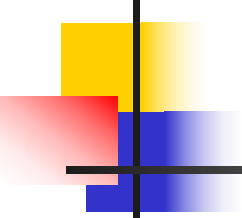
十进制的小数转二进制：乘2取整法

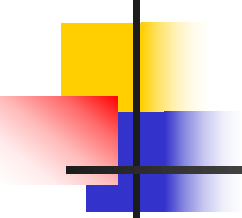
- 
- 尾数所占的位数决定值的精度，指数所占的位数决定值的范围。
 - **float**占**4B**，其中符号**1b**，指数**8b**，尾数**23b**，其精度大约为**7**位，范围约 **$10^{-38} \sim 10^{+38}$** 。
 - **double**占**8B**，其中符号**1b**，指数**11b**，尾数**52b**，其精度大约为**15**位，范围约为 **$10^{-308} \sim 10^{+308}$** 。
 - 很多编译器将**long double**处理为**double**，在某些系统中，它占用**10**或**12B**。很少被使用。

浮点数的可表示误差

■ $5.1 = 101.\underline{0011}\underline{0011}\underline{0011}...$
 $= 1.010\ 0011\ 0011\ 0011... \times 2^2$

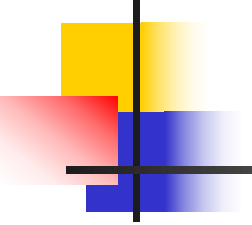
0	10000001	01000110011001100110011
---	----------	-------------------------

- 
- 浮点数的表示可能只是近似的。其值与表示法之间的差称为“可表示误差”。
 - 计算也可能造成可表示误差。不能使用==和!=运算符比较浮点数据。
 - 可以用两个数值之差同一个小正数 **epsilon** 比较的方法解决这个问题。



浮点数溢出的处理

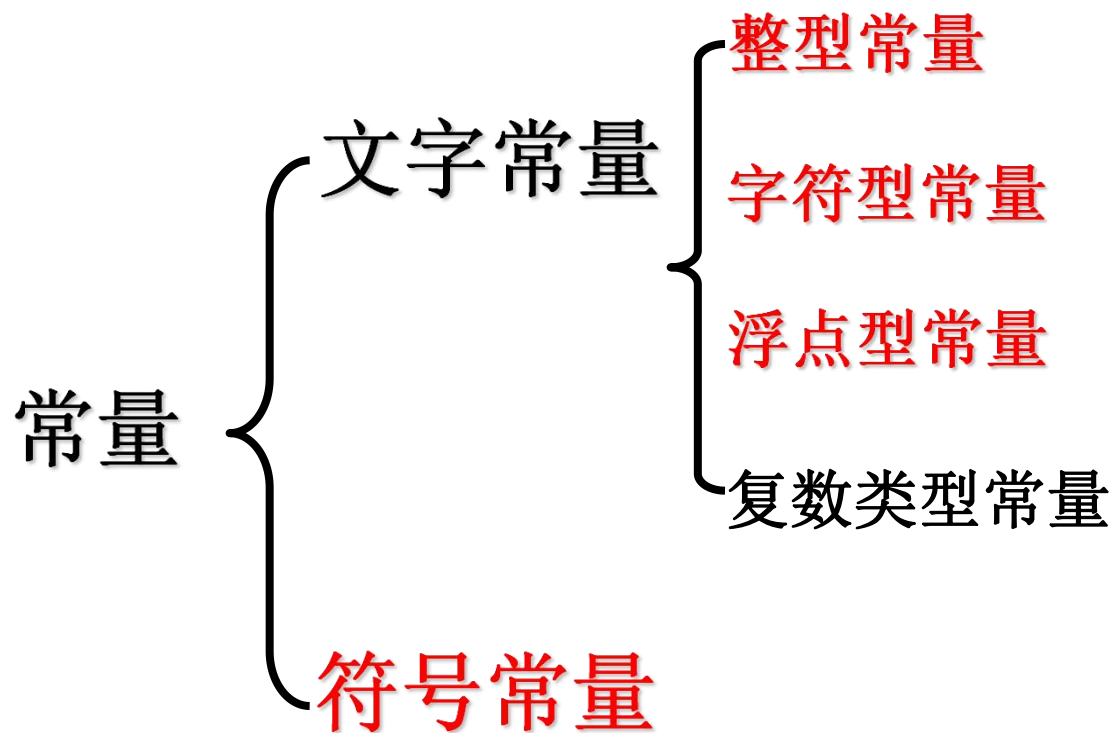
- 下溢时，保存一个非规则化数值，即指数域全为0，对应的阶码E是 126而不是 127，尾数域非0，对应的尾数M小于1且大于0，因此，不再加上高位的1，而是还原为0.xxxxxx，这样做是为了表示0，以及接近于0的很小的数字。当指数域和尾数域全0，这个数是0。
- 上溢时，用称为“无穷大”的特殊位模式表示，即指数域全为1，尾数域0。有些系统中将输出+Infinity或 Infinity表示上溢的数据；指数域全为1，尾数域非0，表示这个数不是一个数（NaN）。程序员必须能识别本地编译器和系统的上溢和下溢值，能识别并修正造成溢出的错误运算。

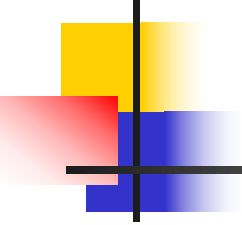


布尔类型_Bool

- C语言中可以用任何整数类型表示布尔值，0表示假，所有非0表示真。
- C99标准新增_**Bool**类型，长度为1，值为0和1（表示“**false**”与“**true**”），使用_**Bool**类型更清晰。
- 为了让C和C++兼容，C99增加了头文件**stdbool.h**，里面定义宏名**bool**为_**Bool**的同义词，定义**false**与**true**分别为0和1。
- 将任意非零值赋给_**Bool**类型变量，都会先转换为1，表示真。

2.4 常量与变量



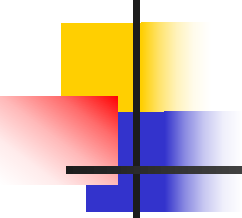


整型常量

有三种表示方法（通过前缀字符区分）：

- 十进制：无前缀
- 八进制：前缀为0
- 十六进制：前缀为0x或0X时。

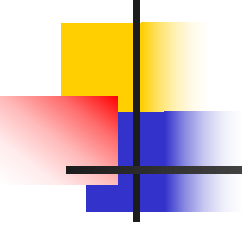
例如，31可写成037，也可写成0x1f或0X1F



整型常量的后缀

用以指定其类型

- 字母**u或U**表示**unsigned**
- 字母**l或L**表示**long**
- 字母**ul或UL**表示**unsigned long**
- 字母**ll或LL**表示**long long (C99)**
- 字母**ull或ULL**表示**unsigned long long (C99)**
- 无后缀时，一般表示**int**
- 当常量值超出指定类型的范围时，其实际类型取决于数值大小、前缀等，确定类型的规则很复杂，在标准化前的**C语言、C89和C99**中各不相同



浮点型常量

有两种表示方式:

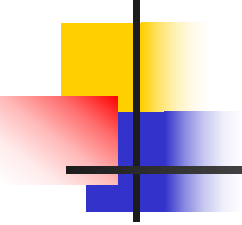
(1) 带小数点的**十进制数**形式 (可以小数点开头, 也可以小数点结尾)

如 **23.7**, **14.**, **.126**

(2) **指数形式** (科学计数法)

将指数部分跟在尾数部分后面。尾数部分的书写规则与第一种相同, 但可以没有小数点, 指数部分 $e(E) \pm n$, 代表 $10 \pm n$ 。

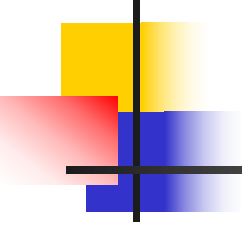
如 **45e-3** = 45×10^{-3} , **.15e5** = 0.15×10^5 。



浮点型常量的后缀

可以使用**后缀**来指定其类型

- 无后缀: **double**
- 后缀**f或F**: **float**
- 后缀**l或L**: **long double**



字符常量

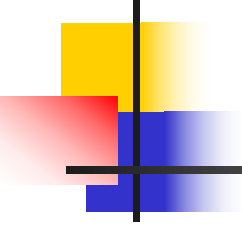
(1) 用单引号包含的一个字符是字符常量

(2) 只能包含一个字符



‘a’ , ‘A’ , ‘1’

‘abc’ 、 “a” ❌



转义序列

■ 以\开头的特殊字符称为转义序列,有两种形式:

➤ 一种是“**字符转义序列**”，反斜线后面跟一个图形符号，用于表示字符集中的非图形符号和一些特殊的图形字符。

\n 换行

\t 水平制表符

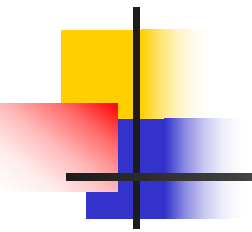
\\ 反斜杠

\' 单引号

\“ 双引号

\0 空字符

➤ 另一种是“**数字转义序列**”



转义序列的应用

```
#include<stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("\n\n causes \na line feed to occur");
```

```
    printf("\n\\\"causes a double quote (\") to be printed");
```

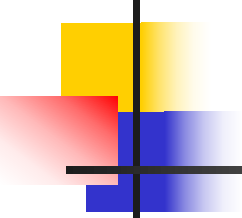
```
    printf("\n\\a causes the bell,or beep, to sound\a");
```

```
    printf("\n\t can be used to align some numbers to tab");
```

```
    printf("columns \n\t1\t2\t3\n\t4\t5\t6");
```

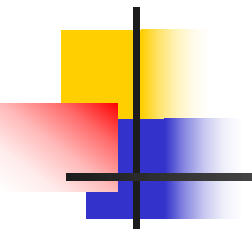
```
    return 0;
```

```
}
```



数字转义序列

- 转义序列的另一种是“数字转义序列”，即
 - \ooo**（1~3个八进制数字）
 - \xhh**（1~2个十六进制数字）
- 例如，
 - 'A'、'\101' 和 '\x41'** 字符**A**
 - '\t'、'\11'、'\011'、'\x9'和'\x09'** 水平制表符



字符串常量

- 写成用一对双引号括住0至多个字符的形式。

"string\n" /* 包含7个字符的字符串 */

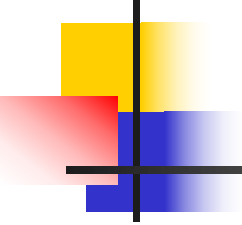
"" /* 包含0个字符的空字符串*/

- 字符串中的单引号可以用图形符号表示，但双引号和反斜线必须用转义序列表示。例如：

"3'40\"" /* 表示5个字符的字符串：3'40" */

"c:\tc" /* 表示4个字符的字符串 */

"c:\\tc" /* 表示5个字符的字符串 */



较长的字符串的写法

有两种方法：

(1) 行连接：在前一行的末尾输入续行符（\）再换行。

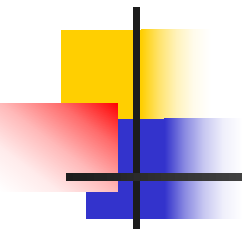
```
"Hello,\n
```

```
how are you" /* 换行后应紧靠行首 */
```

(2) 字符串连接：将字符串分段，分段后的每个字符串用双引号括起来。

```
"Hello, "
```

```
"how are you" /* 换行后不必紧靠行首*/
```



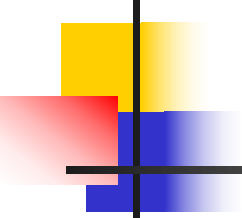
‘a’与“a”的区别

‘a’：字符常量，占1 B内存空间

“a”：字符串常量，占2B内存空间

a	\0
---	----

- 存储时，系统自动在后面补上\0（空字符，ASCII值为0，作为字符串结束标志）
- 字符串的存储长度比字符串的实际长度大1



求字符串的长度

```
#include <stdio.h>
```

```
int main( )
```

```
{
```

```
    char s[20]="world"; /* 用于保存串 */
```

```
    int i;                /* 计数器变量，用于保存串的长度 */
```

```
    i=0;                  /* 计数器变量清零 */
```

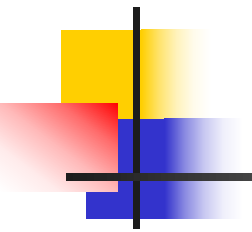
```
    while(s[i] != '\0')
```

```
        ++i;
```

```
    printf( "%d", i);
```

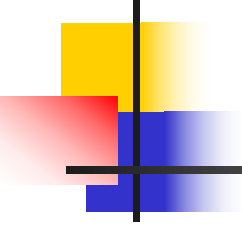
```
    return 0;
```

```
}
```



用函数实现求字符串的长度

```
#include<stdio.h>
int mystrlen(char s[]);    /* 函数原型 */
int main(void)
{
    char s[20]="world";
    printf("%d\n",mystrlen(s)); /* 函数调用 */
    return 0;
}
int mystrlen(char s[ ])    /* 函数定义 */
{
    int i=0;
    while(s[i]!='\0') i++;
    return i;
}
```



符号常量

用一个标识符表示一个常量.

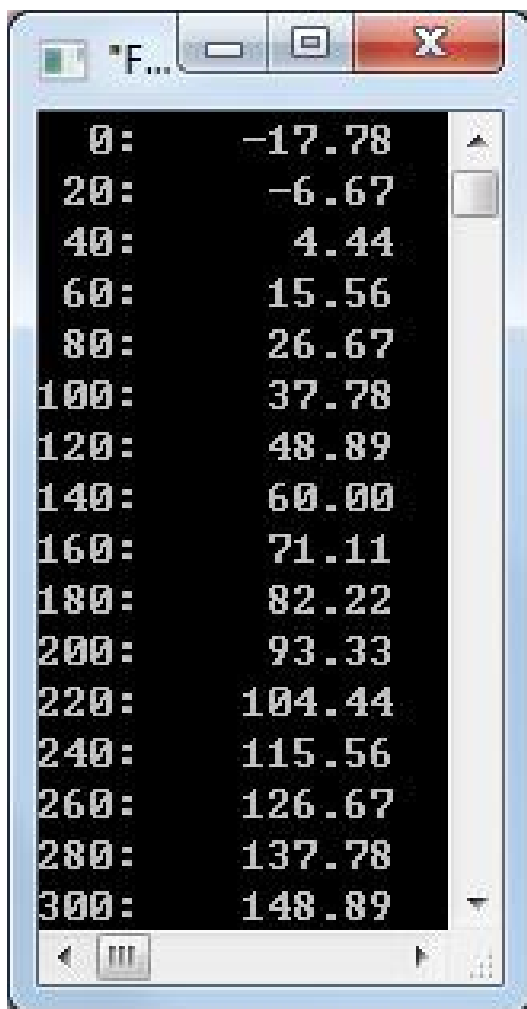
C语言中有三种定义符号常量的方法:

(1) 用**#define**指令 (常用)

(2) 用**const**声明语句 (只读变量、常变量)

(3) 用枚举类型 (在2.9节介绍)

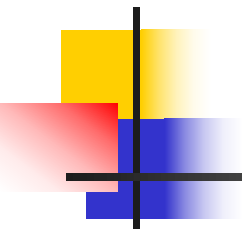
输出华氏和摄氏温度对照表



0:	-17.78
20:	-6.67
40:	4.44
60:	15.56
80:	26.67
100:	37.78
120:	48.89
140:	60.00
160:	71.11
180:	82.22
200:	93.33
220:	104.44
240:	115.56
260:	126.67
280:	137.78
300:	148.89

温度转换公式为:

$$^{\circ}\text{C} = \frac{5}{9} (^{\circ}\text{F} - 32)$$



while循环实现

```
/* print Fahrenheit-Celsius table */
#include <stdio.h>
int main(void)
{
    int fahr;

    fahr=0;
    while (fahr <= 300) {
        printf( "%3d:  %10.2f\n" , fahr, (5.0/9)*(fahr - 32) );
        fahr = fahr + 20;
    }
    return 0;
}
```

符号常量的应用

```
#include <stdio.h>
```

```
#define LOWER 0    /* 表的下限 */
```

```
#define UPPER 300  /* 表的上限 */
```

```
#define STEP 20    /* 步长 */
```

```
int main(void)
```

```
{
```

```
    int fahr;
```

```
    fahr=LOWER;
```

```
    while (fahr <= UPPER) {
```

```
        printf( “%3d: %10.2f\n” , fahr, (5.0/9)*(fahr - 32) );
```

```
        fahr = fahr + STEP;
```

```
    }
```

```
    return 0;
```

```
}
```

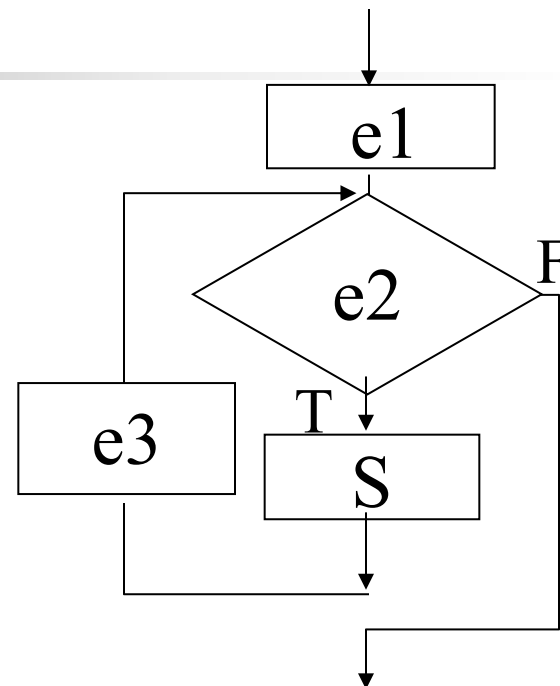
$$^{\circ}\text{C} = \frac{5}{9} (^{\circ}\text{F} - 32)$$

for循环实现

```
#include <stdio.h>
#define LOWER 0    /* 表的下限 */
#define UPPER 300  /* 表的上限 */
#define STEP 20    /* 步长 */
int main(void)
{
    int fahr;

    for(fahr = LOWER; fahr <= UPPER; fahr = fahr + STEP)
        printf( "%3d %10.2f\n" , fahr, (5.0/9)*(fahr - 32) );

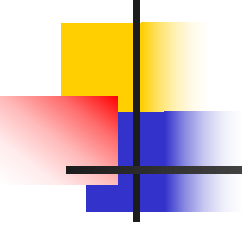
    return 0;
}
```



```
for(fahr = LOWER; fahr <= UPPER; fahr = fahr + STEP)
    printf( "%3d %10.2f\n" , fahr, (5.0/9)*(fahr - 32) );
```

```
return 0;
```

```
}
```

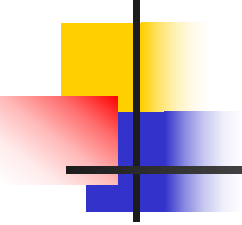


用#define定义符号常量

#define是一种编译预处理指令,格式为:

#define 标识符 常量

↑
符号常量(一般用大写,以区分变量)



用const定义符号常量

const是关键字，称为类型限定符。格式为：

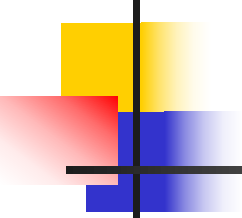
const 类型名 标识符=常量;

例如：

const double PI=3.14159;

const int DOWN=0x5000; /* 下光标键的扫描码 */

const int YES=1, NO=0;

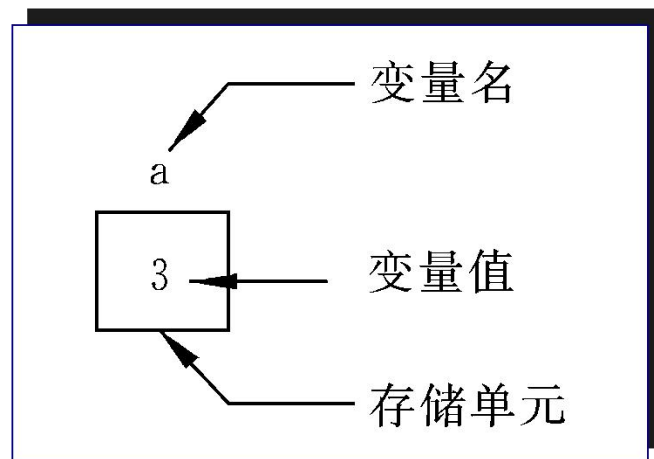


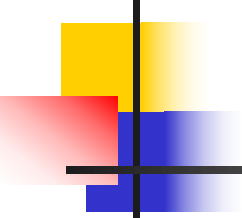
const和#define的区别

- **const**声明的标识符是一个只读变量，编译时系统会根据定义的类型为该标识符分配存储单元，并把对应的常量值放入其中，该值不能再被更改，此后，程序中每次出现该标识符都是对所代表存储单元的访问。
- **#define**定义的标识符没有对应的存储单元，只是在编译之前由预处理程序进行简单的文本替换。

变量

- 变量代表内存中具有特定属性的一个存储单元，它用来存放数据，这就是变量的值，在程序运行期间，这些值是可以改变的。





变量的定义

- 要求对所有用到的变量作定义，也就是“先定义，后使用”

类型名 变量表；

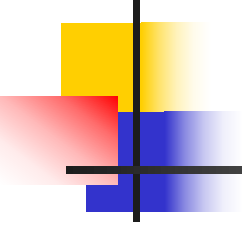
int total, average;

- 变量在声明时可以同时赋一个初值（称为变量的显示初始化），每个变量必须分别显示初始化。

int count=0, sum=0;

char alert='a', c ;

int count=0,sum=0; （不能 **int count=sum=0;**）



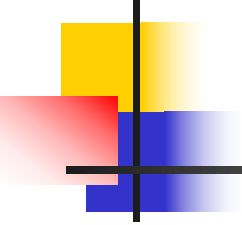
思考与练习

(1) 练习例2.9，了解`%3d`，`%10.2f`等域宽控制，然后将其中的`“(5.0/9)*(fahr - 32)”`改成`“(5/9)*(fahr-32)”`会输出什么结果，思考为什么。

(2) 仿写：实现将字符串反转。

例如 “**hello**” 反转为 “**olleh**”

(3) 仿写：利用**for**循环实现输出**1!**，**2!**，**...**，**10!**的值。



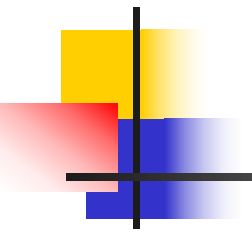
2.5 运算符和表达式

- 运算符是运算的符号表示，执行对运算对象（称为操作数）的各种操作。
- 单个的操作数（包括常量、变量和函数调用）是表达式，由运算符和操作数组成的有意义的计算式子更是表达式，如：

`sqrt(b*b-4*a*c)`

`x=x*PI/180`

`fabs(an)>=EPS`



C运算符简介

(1) 运算符的运算功能

(2) 操作数的个数和类型要求

单目（或一元）运算符

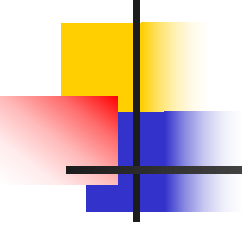
双目（或二元）运算符

三目（或三元）运算符

运算符都对操作数的类型有规定，比如%的操作数不能为浮点型。

(3) 运算所得结果的类型

运算的结果是一个具有确定类型的值，这个类型称为表达式值的类型。尤其当两个不同类型的操作数进行运算时，会引起数据类型的转换，特别要注意结果值的类型。



运算符的优先级和结合性

- 当表达式中包括多个运算符时，C语言会先按优先级规则解释表达式的意义。如：

$1+2*3$ 等价于 $1+(2*3)$

- 当一个操作数两侧的运算符优先级别相同时，则按“结合性”规则。

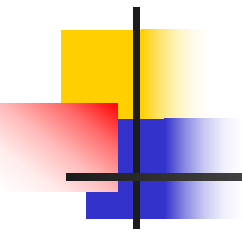
$1+2-3$ 等价于 $(1+2)-3$

---从左至右的结合性（左结合）

$-a++$ 等价于 $-(a++)$

----从右至左的结合性（右结合）

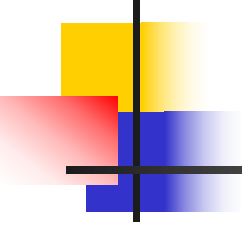
- 所有运算符的优先级和结合性规则见表2-4



算术运算

■ 运算符:

	双目	单目		
+	加法	正值	$3+6$, $+3$	
-	减法	负值	$6-4$, -5	
*	乘法		$3*8$	
/	除法		$8/5$	值为1
%	求余		$7\%4$	值为3



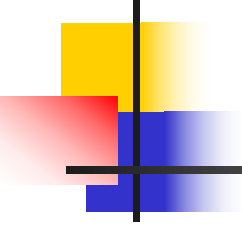
注意

★两个整型数据相除（结果为整, 舍去小数部分）

$$-5/3 \Rightarrow -1 \quad 1/2 \Rightarrow 0 \quad 1./2 \Rightarrow 0.5$$

使用时千万注意 `int / int` 出现数据丢失。

★ `%`操作数必需为整数



求出所有的水仙花数

- “水仙花数”是一个三位数，其各位数字立方和等于该数本身。例如，153是一个“水仙花数”，因为 $153=1^3+5^3+3^3$ 。
- 找出“水仙花数”的关键是怎样从一个三位数中分离出百位数、十位数和个位数。

$i=x/100;$ $/*$ 分解百位数 $*/$

$j=(x-i*100)/10;$ $/*$ 分解十位数 $*/$

$k=x\%10;$ $/*$ 分解个位数 $*/$



输出水仙花数的程序

```
#include<stdio.h>

int main(void)
{ int x,i,j,k;
  for(x=100;x<=999;x++) { /* 枚举每一个3位数 */
    i=x/100;              /* 分解百位数 */
    j=(x-i*100)/10;      /* 分解十位数 */
    k=x%10;              /* 分解个位数 */
    if(x==i*i*i+j*j*j+k*k*k) /* 是水仙花数 */
      printf("%5d",x);
  }
  return 0;
}
```



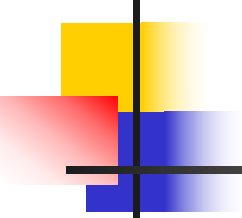
水仙花数程序的函数版

```
#include<stdio.h>

int main(void)
{ int x;
  for(x=100;x<=999;x++) {
    if( isNarcissus(x) ) /*x是水仙
      printf("%5d",x);
    }
    return 0;
  }
```

```
int  isNarcissus (int m)
{
    int i,j,k;
    i=m/100;
    j=(m-i*100)/10;
    k=m%10;
    if(m==i*i*i+j*j*j+k*k*k)
        return 1;
    else return 0;
}
```

函数**isNarcissus**: 判断一个数是，返回1，不是返回0。



关系运算

有6个关系运算符：

> 大于

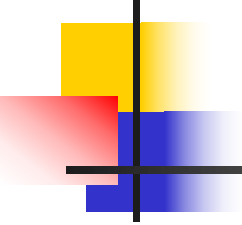
>= 大于等于

< 小于

<= 小于等于

= 等于

!= 不等于



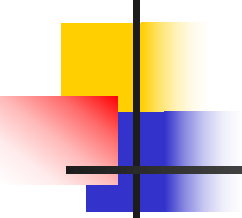
常见的编程错误

- 将运算符**`==`**写成运算符**`=`**（赋值）。可能会因为运行时的逻辑错误而导致不正确的结果。例如，

```
if (grade == 'A') printf("Very Good! ");  
/*当成绩的等级为A等时，输出Very Good!*/
```

而：

```
if (grade ='A') printf("Very Good!");  
/*不论成绩的等级是几等，总输出Very Good!*/
```



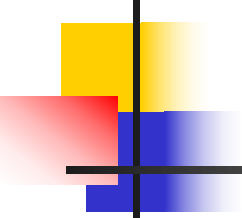
关系表达式的类型和值

- 关系表达式的类型: **int**

- 关系表达式的值:

关系成立, 值为**1** (代表“真”)

关系不成立, **0** (代表“假”)



关系表达式举例

根据变量说明，给出表达式的值。

int x=4, y=3, z=2;

char c='a';

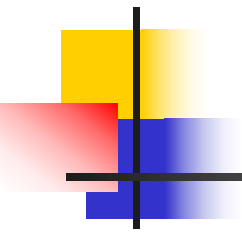
(1) **c == 'A'+32** (1)

(2) **c+1 != 'b'** (0)

(3) **x-y<=10** (1)

(4) **z=x>y** (1)

(5) **x>y>z** (0)



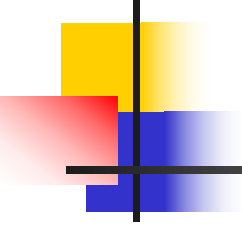
注意

- 数学上判断**x**是否在区间**[a,b]**中时，习惯写成：

$$a < x < b$$

- C中“**a < x < b**”的含义与数学中的含义不同，
应写成：

$$a < x \ \&\& \ x < b$$



逻辑运算

- 有3个逻辑运算符

!

逻辑非

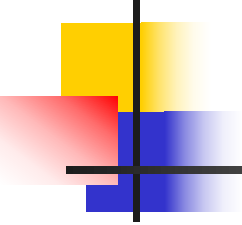
&&

逻辑与

||

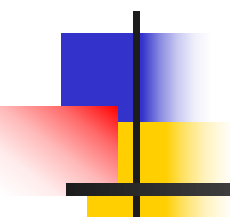
逻辑或

逻辑表达式：用逻辑运算符将关系表达式
或逻辑量连接起来的式子



逻辑表达式的类型和值

- 逻辑运算的操作数可以是**0**和任何非**0**的数值，
- 系统最终以**0**判断属于“**假**”（以**0**代表“假”），
以**非0**判断属于“**真**”（以**1**代表“真”）。
- 逻辑表达式的值：**1**（“真”）
0（“假”）
逻辑表达式的类型：**int**

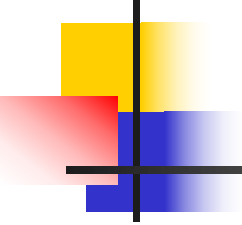


&& 和 || 的真值表

e1	e2	&&	
0	0	0	0
0	非0	0	1
非0	0	0	1
非0	非0	1	1

! 的真值表

e	!
0	1
非0	0



写表达式

(1) 整数a是偶数

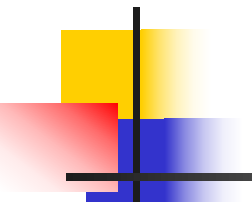
$!(a\%2)$ 或 $a\%2==0$

(2) 字符c的值是英文字母。

$c>='a' \ \&\& \ c<='z' \ || \ c>='A' \ \&\& \ c<='Z'$

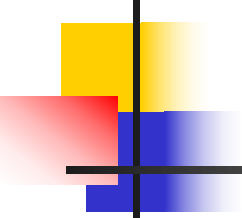
(3) 某一年year是闰年。如果某一年的年份能被4整除但不能被100整除，那么这一年就是闰年，此外，能被400整除的年份也是闰年。

$!(year\%4) \ \&\& \ year\%400 \ || \ !(year\%400)$



判闰年的程序

```
#include<stdio.h>
/* 判断闰年,year 是闰年, 返回1; 否则, 返回0 */
int isleap (int year)
{
    if ( !(year%4) && year%400 || !(year%400) )    return 1;
    else      return 0;
}
int main(void)
{ int year;
  scanf("%d",&year);
  if ( isleap(year) ) printf("%d is a leap year\n",year);
  else printf("%d is not a leap year\n",year);
  return 0;
}
```



注意

- 编译程序在处理含有**&&**、**||** 表达式时，往往采用优化算法(提高速度)。

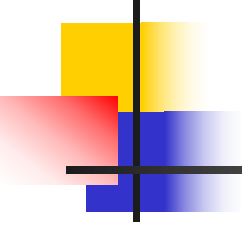
e1&&e2 一旦发现**e1=0**，不再计算 **e2**

e1 || e2 一旦发现**e1=1**，不再计算 **e2**

- 例如，

x>=0.0 && sqrt(x)<=7.7

/* 如果x值为负，不求x的平方根 */



自增和自减运算

- 只能用于变量

++: 自增, 使变量值加1

--: 自减, 使变量值减1

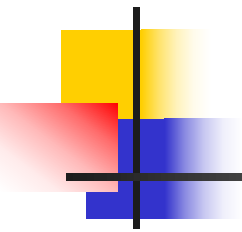
- 奇特之处: 有前缀式和后缀式

++x

x++ 相当于 **x=x+1;**

--x

x-- 相当于 **x=x-1;**



前缀与后缀的区别

$x = 10 ;$

$y = ++x ; \quad /* y=11, x=11 */$

---- 先执行对操作数的加运算，
再使用该操作数的值

$y = x++ \quad /* y=10, x=11 */$

---- 先使用该操作数的值，
再对它作加运算，

应用

//计算 $1+2+3+\dots+n$

```
#include<stdio.h>
```

```
int main(void)
```

```
{
```

```
    int i,sum,n;
```

```
    scanf("%d",&n);
```

```
    i=1; sum=0;
```

```
    while(i<=n) {
```

```
        sum=sum+i;
```

```
        i++;
```

```
    }
```

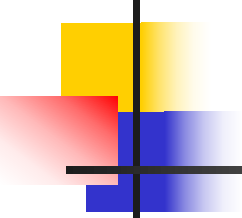
```
    printf("sum=%d\n",sum);
```

```
    return 0;
```

```
}
```

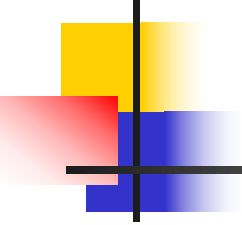
简化代码

sum=sum+i++;



序列点

- 程序执行中的点。变量值的改变必须在序列点之前，完成，可以保证在序列点之后，是更改后的新值。
下列条件出现序列点：
- (1) **&&**、**||**、**?:** 或 **,** 运算符，即这些运算符的第一个操作数之后
- (2) **完整表达式结束**时，即表达式语句、**return**语句中的表达式、**if**、**switch**或循环语句中的条件表达式（包括**for**语句中的每个表达式）之后



后缀式举例

int a=1,b=0;

(1) b++ + b++

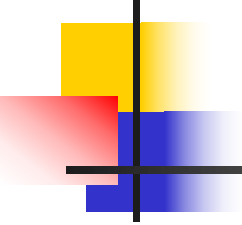
值不确定

(2) a--&& a

表达式值为0， a为0

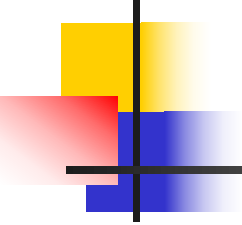
(3) b++ ? b : -b

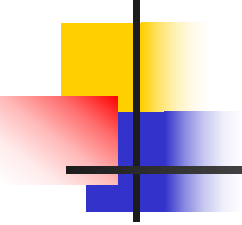
表达式值为-1， b为1



表达式的副作用

- **表达式的副作用**是指：表达式在求值过程中要改变该表达式中作为操作数的某个变量的值。
- **具有副作用的运算符**：赋值、复合赋值、增1、减1
- **c标准未规定函数参数的求值顺序，也没有规定除了 &&、|| 和 ， 外的双目运算中两个操作数的计算顺序。由编译器自行规定计算顺序。**
- 由于编译器对求值顺序的不同处理，当表达式中带有副作用的运算符时，就有可能产生二义性。

- 
- 要慎重使用自增、自减运算符
 - 在一个表达式中不要多处出现变量的自增、自减等运算。
 - 尽量不要在函数参数中用自增减表达式



统计正文的字符数和行数

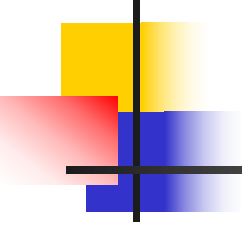
- 正文是一行行字符组成的字符序列，每一行是以换行符为结束标志的一串字符。

Upload file ✓

Special pages ✓

Page information ✓

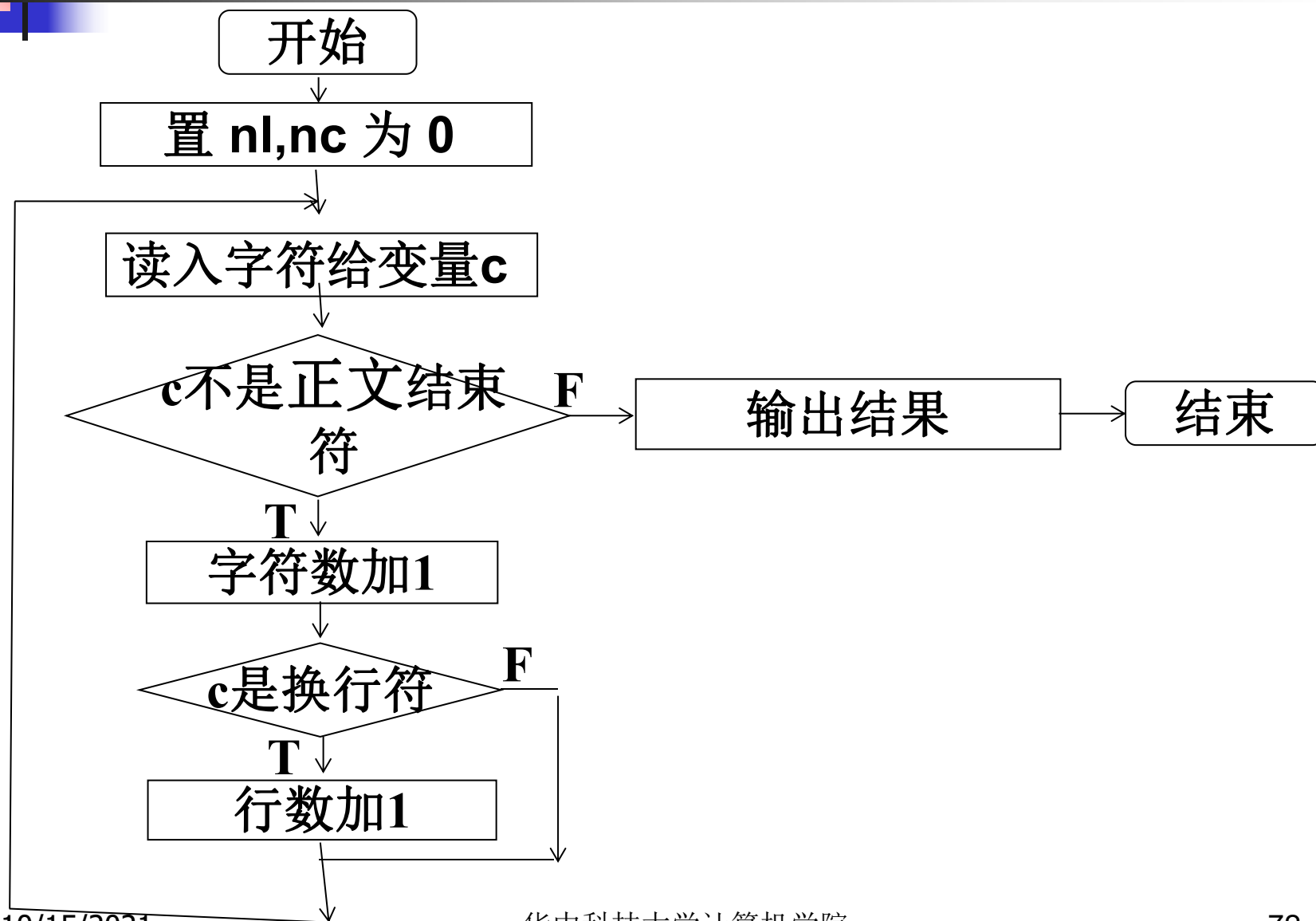
Ctrl+z ✓



文件结束标志

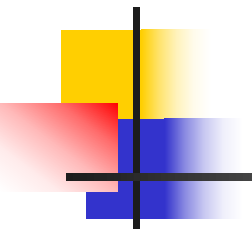
- 输入正文以**Ctrl+Z**（DOS系统）或**Ctrl+D**（UNIX系统）为结束标志（称为文件尾）
- 字符的输入可以使用标准库函数**getchar**或**scanf**，遇文件结束标志时返回**EOF**，
- **EOF**是在头文件**<stdio.h>**定义的符号常量，其值为**-1**。

流程图



程序

```
#include<stdio.h>
/*统计输入正文的字符数和行数。*/
int main(void)
{
    char c;
    int nc,nl;
    printf("Input a text end of ctrl+z:\n"); /* 提示语 */
    nc=nl=0;
    while((c=getchar())!=EOF)
    {
        ++nc; /* 字符数自增1, 可以用nc++代替 */
        if(c=='\n') ++nl; /* 行数自增1, 可以用nl++代替 */
    }
    printf("nc=%d,nl=%d\n",nc,nl);
    return 0;
}
```



思考与练习

- **思考：**将例2.15程序中的while循环改为for循环
 - **练习：**
 - 1) 统计输入正文中水平制表符、空格、换行符的个数
 - 2) 把输入的一行字符复制到屏幕输出，但是前导空格不输出。
- 均要求用getchar函数输入字符

赋值运算

1. 简单的赋值运算符 (=)

赋值表达式一般形式为: <变量> = <表达式>

左值 (lvalue): 赋值运算符左侧的标识符
变量可以作为左值

而表达式、常量不能作为左值(如 $a+b$)

2. 复合的赋值运算: 在“=”号之前加一个双目运算符

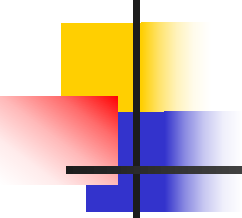
$+=, -=, *=, /=, \%=$

$i+=2$ 等价于 $i=i+2$

$y/=x+10$ 等价于 $y=y / (x+10)$

$x*=k=m+5$ 等价于 $x=x * (k=m+5)$

$s[i++] += 1$ 和 $s[i++] = s[i++] + 1$ 不等价



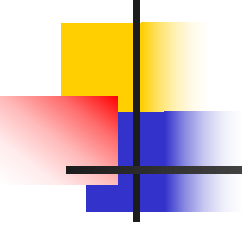
赋值表达式的值和类型

- 与左操作数的值和类型相同

int x;

x=5.6; // 等价于 x= (int) 5.6;

- 强制类型转换运算符
(类型名) 操作数



赋值运算符的结合性

- 右结合性，如

a=b=c=3;

等价于

a= (b= (c=3)) ;

- C把赋值处理为运算符，其好处是使赋值表达式可以像其它任何表达式一样当作一个数据来处理。 如

a=2;

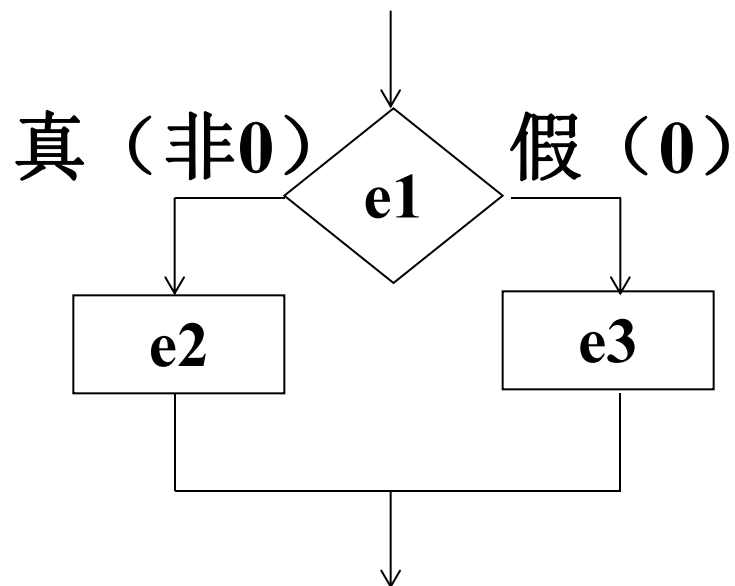
b=3;

x=a+b; 可以被简化为

x= (a=2) + (b=3)

条件运算

■ $e1 ? e2 : e3$



if(e1)

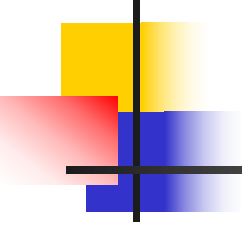
值=e2;

else

值=e3;

■ $x=10;$

$y=x>9?100:200$ */* y: 100 */*



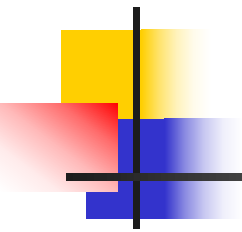
嵌套的条件表达式

$$y = \begin{cases} 1 & x > 0 \\ 0 & x = 0 \\ -1 & x < 0 \end{cases}$$

$$y = x > 0 ? 1 : x < 0 ? -1 : 0$$

等价于(因为是右结合性,括号可省略)

$$y = x > 0 ? 1 : (x < 0 ? -1 : 0)$$



练习

- (1) 一个数如果恰好等于除它本身外的所有因子之和，这个数就称为“完数”。例如 $6=1+2+3$ ，则6是完数。
- (2) 改写例2.8, 统计字符串中十六进制数字字符的个数。
- (3) 改写例2.15, while循环改为for循环。
- (4) 练习例3.1, 3.2, 3.3, 3.4, 熟悉getchar和putchar用法。
- (5) 用getchar输入一段正文，将输入中的双引号（”）和反斜线（\）转换成对应的转义序列（\”，\\）输出, 其他字符原样输出。字符输出用putchar。例如，

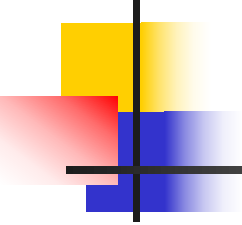
输入 123” as

 \world

 ^Z

输出 123\” as

 \\world



逗号运算

e1, e2

- 计算规则:

先计算**e1**，再计算**e2**，逗号表达式的值和类型与**e2**的值和类型相同

- **x=(i=4, i%3)**

表达式的值为：1， **x**为1

- **x=i=4, i%3**

表达式的值为：1， **x**为4

应用

//计算 $1+2+3+\dots+n$

```
#include<stdio.h>
```

```
int main(void)
```

```
{
```

```
    int i,sum,n;
```

```
    scanf("%d",&n);
```

```
    sum=0;
```

```
    for(i=1;i<=n;i++) {
```

```
        sum+=i;
```

```
    }
```

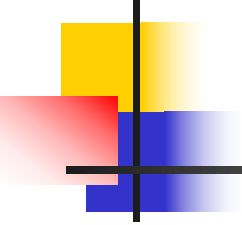
```
    printf("sum=%d\n",sum);
```

```
    return 0;
```

```
}
```

空语句

```
for(sum=0,i=1;i<=n;sum+=i,i++)  
;
```



扩展形式

逗号表达式的一般形式可以扩展为

$e_1, e_2, e_3, \dots, e_n$

它的值为表达式 e_n 的值。

注意

并不是任何地方出现的逗号都是作为逗号运算符。例如函数参数也是用逗号来间隔的。

```
printf(“%d,%d”, 2, 3); /* 输出: 2, 3 */
```

```
printf(“%d,%d” ( 2, 3), 3 ); /* 输出: 3, 3 */
```

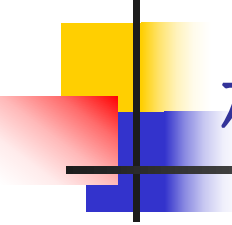
“2, 3”并不是一个逗号表达式，它是printf函数的2个参数

“ (2, 3) ”是一个逗号表达式，它的值等于3。

反转字符串

```
i=0;    // 首字符位置
j=strlen(s)-1; // 最后一个字符位置 */
while(i<j) {
    mid=s[i];
    s[i]=s[j];
    s[j]=mid; /*交换s[i]、s[j] */
    i++;
    j--;    /* i、j向中间靠拢 */
}
```

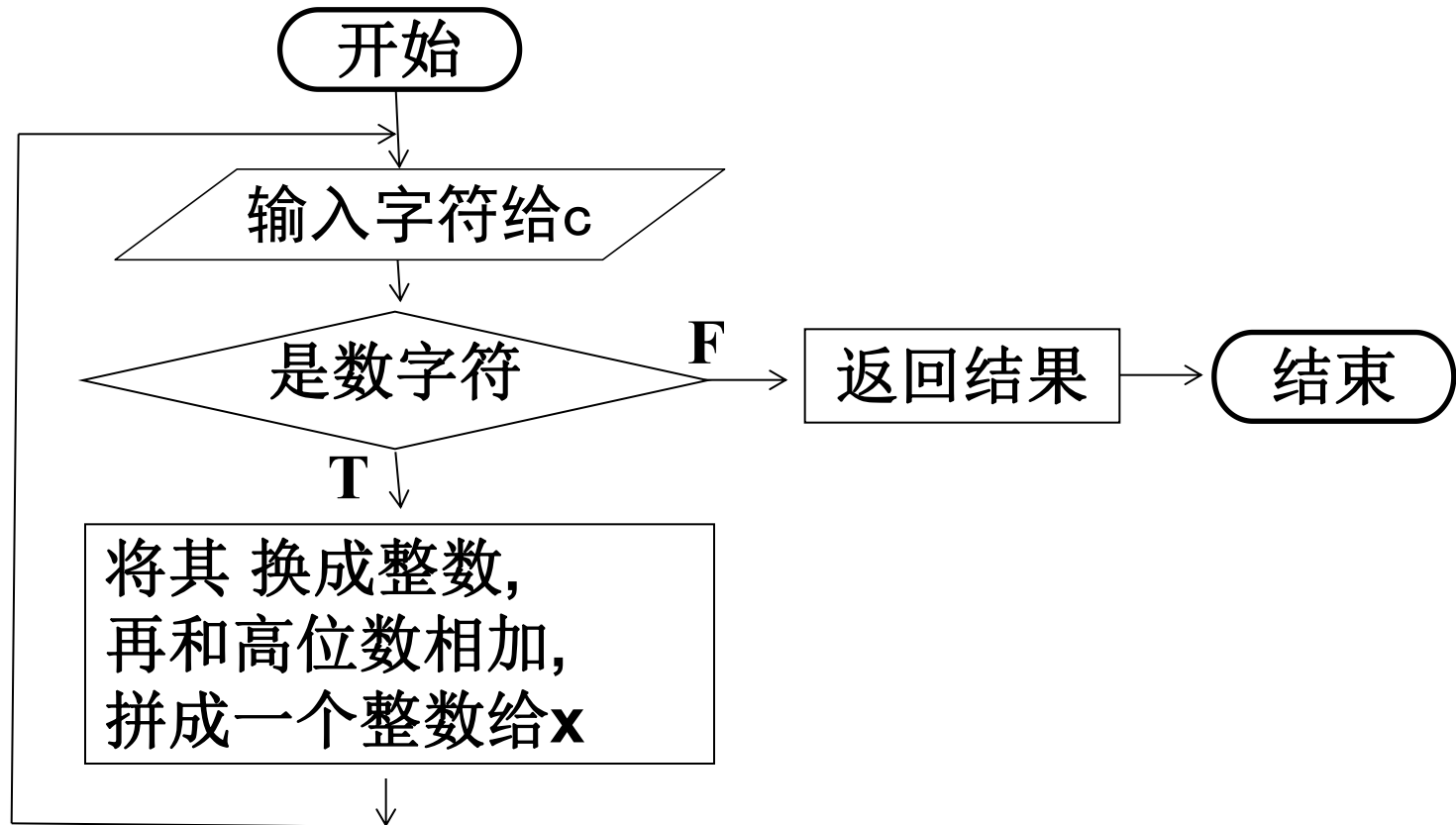
```
for(i=0,j=strlen(s)-1;i<j;i++,j--)
    mid=s[i],s[i]=s[j],s[j]=mid;
```



模拟实现scanf("%d",&x) 的功能

- getchar 实现了scanf("%c",&c) 的功能
- 实际上输入的是数字符，scanf函数读到的每个数字都是该字符的字符码。
- 数字字符转换成对应的整数
 $c - '0'$
- 循环拼数
 $x = x * 10 + c - '0'$

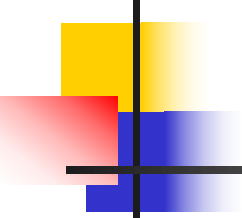
流程图



程序

```
int getint(void)
{
    char c;    /* 用于存放当前输入的字符 */
    int x;     /* 用于存放转换的整数 */
    for(x=0, c=getchar(); c>='0' && c<='9'; c=getchar()) /* 遇非数字字符结束 */
        x=10*x+c-'0';    /* c-'0': 将数字字符转换成对应的一位整数 */
    return x;
}

int main(void)
{
    int score;
    score=getint();
    printf("%d\n", score);
    return 0;
}
```

sizeof运算

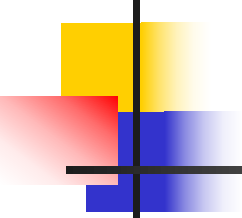
一个单目运算符，有两种形式：

(1) sizeof (类型名)

给出指定数据类型占用的存储字节数

(2) sizeof 表达式

给出表达式结果的类型占用的存储字节数



举例

假设int类型占用2字节。

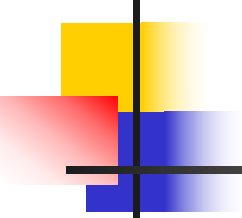
■ **sizeof(long)** /* 值为4 */

■ **double x; sizeof x** /* 值为8 */

■ **short a[10]; sizeof (a)** /* 值为20 */

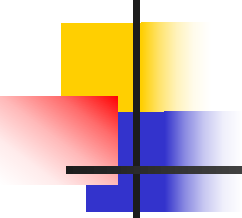
■ **int a=1,b=1; sizeof (a+b)**

/* 值为2, 而sizeof a+b, 先求sizeof a, 再和b
加, 值为3 */



sizeof是一个常量表达式

- 其运算是在编译时执行的。因此，当**sizeof**的操作数是表达式时，则在编译时分析表达式以确定类型，运行时不对这个表达式求值。
- 例如：
- `short x=1;`
- `sizeof(++x);` */* x不递增，仍为1 */*



2.6 位运算

6个位运算符:

$\sim$ (求反)

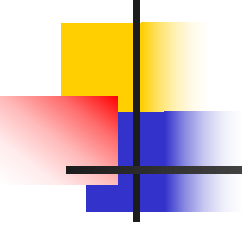
$\&$ (按位与)

$|$ (按位或)

$\wedge$ (按位异或, 或按位加)

$\ll$ (左移)

$\gg$ (右移)

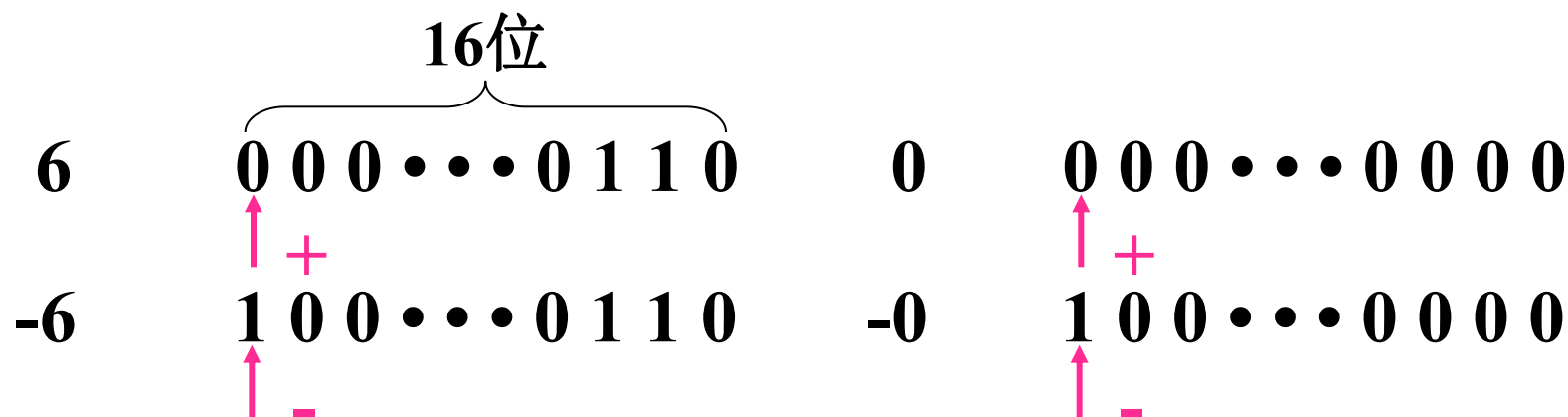


数的机器码表示

- **机器数**：在计算机中，将一个数的最高位定义为符号位，其余各位为数值位。并且规定**符号位之值为0表示正，符号位之值为1表示负**。用这种方法表示的数称为机器数。
- **真值**：机器数的数值称为该机器数的真值。也就是正、负号后跟二进制数的绝对值就构成真值。

原码

最高位为符号位，其余各位为该数的绝对值。



0的原码表示不惟一

反码

- 正数：同原码
- 负数：最高位为1，其余各位对原码取反

6	0 0 0 . . . 0 1 1 0
	↑ +
-6	1 1 1 . . . 1 0 0 1
	↑ -
0	0 0 0 . . . 0 0 0 0
	↑ +
-0	1 1 1 . . . 1 1 1 1
	↑ -

补码

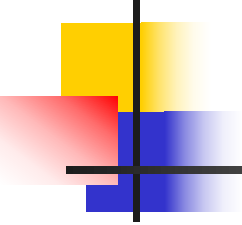
正数：同原码

负数：反码 + 1

$$\begin{array}{r} 6 \quad 000 \cdots 0110 \\ -6 \quad 111 \cdots 1001 \\ \quad \quad \quad +1 \\ \hline 111 \cdots 1010 \end{array}$$

$$\begin{array}{r} 0 \quad 000 \cdots 0000 \\ -0 \quad 111 \cdots 1111 \\ \quad \quad \quad +1 \\ \hline 000 \cdots 0000 \end{array}$$

计算机是以补码形式存放数据的
(钟表原理)



按位求反 ($\sim$)

- 对操作数的每个二进制位取相反值，例如：

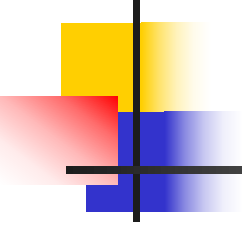
short a=5; unsigned short b=5;

a和b的二进制为： 00000000 00000101

$\sim$ a的二进制为： 11111111 11111010

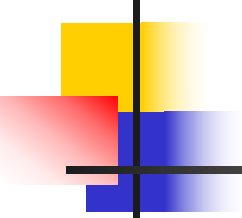
$\sim$ a的值为short型 -6。

$\sim$ b的值为unsigned short型 65530。



按位与(&)、或(|)、加(^)运算

表达式	二进制表示	十六进制值	说明
x	01101000 11010001	0x68d1	待处理数据
mask	11111111 00000000	0xff00	逻辑尺
x & mask	01101000 00000000	0x6800	将x的低字节置为0， 保留高字节
x mask	11111111 11010001	0xffd1	将x的低字节保留， 高字节置为1
x ^ mask	10010111 11010001	0x97d1	将x的低字节保留， 高字节翻转



简单应用

■ 判断一个整数是奇数还是偶数

二进制的最末位为0表示该数为偶数，最末位为1表示该数为奇数。

```
if(!(x&1) ) printf(“%d 是偶数”,x);    // x&1 ==0
```

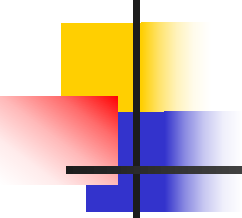
■ 大小写转换

```
c^=32
```

■ 简单的加密

比如你想对某人说21，但不想让别人知道，于是约定用其生日18作密钥（Key）。

$21 \wedge 18 = 7$ ，把7告诉Ta。Ta再次计算 $7 \wedge 18$ ，得到21。



左移(<<)和右移(>>)

- **$e \ll n$** 将**e**的值向左移**n**位，低**n**位填入**0**。
左移1位相当于该数值乘以2
- **$e \gg n$** 将**e**的值向右移**n**位，而高**n**位可能填入**?**。
 - e**是无符号类型，填入**0**；
 - e**是有符号类型，一些机器填入**0**（即“逻辑移位”），
而另一些机器则填入符号位（即“算术移位”）。
- 在使用右移运算符时应经常用无符号类型。
- 右移一位相当于该数值除以2

移位运算符的例子

表达式	二进制表示	值	行为
a	00000000 00001011	11	
b	00000000 00001111	15	
-a	11111111 11110101	-11	对a进行负号运算
a<<2	00000000 00101100	44	a左移2位
b>>3	00000000 00000001	1	b右移3位
-a>>2	11111111 11111101	-3	-a右移2位(填入符号位1, 而其他机器可能是0)

写表达式

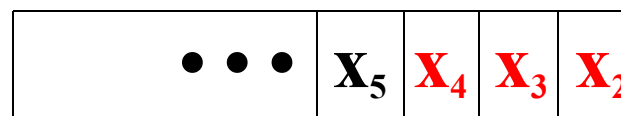
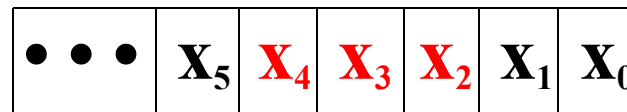
将整数x的第m位起的右n位取出来，作为另一个数

注：一个整数的各个二进制位从右至左依次编号为第0位、第1位、第2位、.....

如： $m=4$ $n=3$

- ① 将要取出的那n位移到最右端

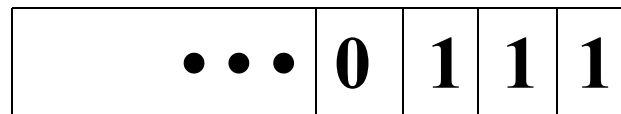
$$x \gg m-n+1$$



- ② 设计一个逻辑尺：低n位全为1，其余位全为0

$$\sim(\sim 0 \ll n)$$

$$(1 \ll n) - 1$$



- ③ 将上面二者进行&的运算

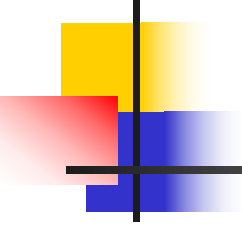
$$x \gg (m-n+1) \& \sim(\sim 0 \ll n)$$

位运算应用----压缩

- 可以把表示21世纪日期的日、月和年3个整数压缩成一个16位的整数(short)。写一个表达式，实现压缩存储日、月和年。
- 因为日有31个值，月有12个值，年有100个值，所以可以在一个整数中用5b表示日，用4b表示月，用7b表示年

15	11 10	7 6	0
日	月	年	

$\text{day} \ll 11 \mid \text{month} \ll 7 \mid \text{year}$



压缩函数pack

函数名: **pack**

输入参数: **day-日**

month-月

year-年

返回值: 压缩后的整数

```
short pack(short day, short month, short year)
{
    return(day<<11 | month<<7 | year) ;
}
```


压缩存储日、月和年

```
short pack(short day, short month, short year); //函数声明语句
```

```
int main()
```

```
{ short day, month, year, date;
```

输入/出 short 数据 用 %hd

输入/出 unsigned short 用 %hu

```
printf("Input year, month, day\n");
```

```
scanf("%hd%hd%hd", &year, &month, &day);
```

```
date = pack(day, month, year); //函数调用
```

```
printf("%hx\n", date);
```

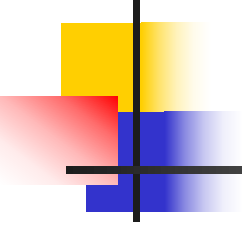
```
return 0;
```

```
}
```

```
short pack(short day, short month, short year) //函数定义
```

```
{ return(day<<11 | month<<7 | year);
```

```
}
```



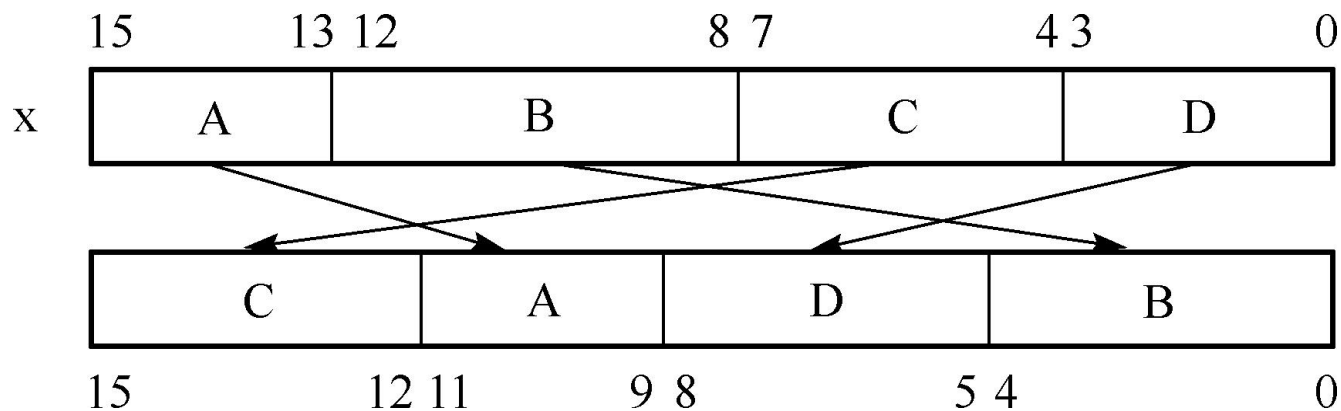
练习：编写解压函数

```
void unpack(short x)
{
    将x解压为 day, month, year
    printf :  day, month, year
}
```

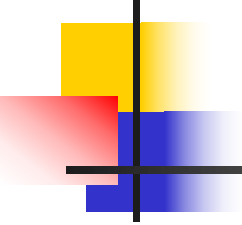
位运算应用----加密

将一个短整型数 x 分成4个长度不等的部分：

A (3b)、**B (5b)**、**C (4b)** 和 **D (4b)**，然后将它们按照**CADB**的顺序重新拼凑在一起，实现对其加密的功能。写一个表达式，要求其值为 x 的密文。



$(x \& 0xE000) \gg 4 \mid (x \& 0x1F00) \gg 8 \mid (x \& 0xF0) \ll 8 \mid (x \& 0xF) \ll 5$



加密函数

函数名: **encrypt**

函数参数:

x-待加密短整数

函数返回值:

加密后的短整数

```
#define AE  0xE000    /* 取出A的逻辑尺 */
#define BE  0x1F00    /* 取出B的逻辑尺 */
#define CE  0x00F0    /* 取出C的逻辑尺 */
#define DE  0x000F    /* 取出D的逻辑尺 */

unsigned short encrypt(unsigned short x)
{
    unsigned short a,b,c,d;
    a=(x&AE)>>4
    b=(x&BE)>>8
    c=(x&CE)<<8
    d=(x&DE)<<5
    return (a|b|c|d);
}
```

计算ASCII码字符的奇偶校验位

二进制数据在传送中会发生**误码**（1变成0或0变成1），这就有如何发现及纠正误码的问题。解决方法是在原始数据（数码位）基础上增加几位校验（冗余）位。

奇偶校验：分奇校验和偶校验。

奇校验：整个码字中奇数个“1”

偶校验：整个码字中偶数个“1”

data='A'



校验位 parity

校验位在每个字符的最高位

```
int main()
```

```
{ unsigned char data,backup,t;
```

```
int parity=0; /* 奇偶校验位,0-偶校验, 1-奇校验 */
```

```
data=getchar(); /* 被校验的数据 */
```

```
backup=data;
```

```
while (data) {
```

```
    t=data&1; /* 取该数的最低位 */
```

```
    parity^=t; /* 进行异或操作 */
```

```
    data>>=1; /* 为取下一位做准备 */
```

```
}
```

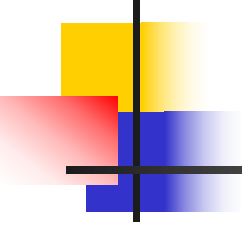
```
data=backup|(parity<<7); /* 生产校验码 */
```

```
printf("The data is %#x\n",backup);
```

```
printf("Parity-Check Code is %#x\n",data); /* 输出校验码 */
```

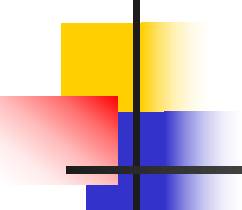
```
return 0;
```

```
}
```



打印整数各位的软件工具

```
/* 按位（二进制）显示 int */
#include<limits.h>
void bit_print(int x)
{
    int i;
    int n=sizeof(int) * CHAR_BIT;    /* CHAR_BIT 在limits.h中定义 */
    int mask = 1 << (n-1);           /* 逻辑尺mask=100.....0 */
    for ( i=1; i<=n; ++i ) {
        putchar ( ! ( x & mask ) ? '0': '1');
        x<<=1;
        if ( ! ( i % CHAR_BIT ) && i<n ) putchar(' '); // 每字节间输出空格
    }
}
```



自主练习

1、练习例3.8, 3.12, 掌握short、long、double类型数据的输入/出方法。

2、编写压缩存储日、月和年的函数，再编写相应的解压函数，在main中测试这两个函数的功能。

```
short pack(short day, short month, short year);  
void unpack(short x);
```

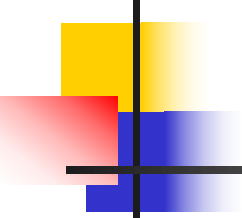
3、自学例2.25, 并编写该加密函数

```
short encrypt(short x); /*形参是明文, 返回密文*/
```

5、研读例2.27, 用本章后习题2.24的方法重写

5、研读例2.26, 仿写: 定义函数hex_print(int x)以十六进制输出int数, 即模拟实现 printf ("%x", x)。

在main中验证: 分别调用hex_print和printf以十六进制输出一个int数。

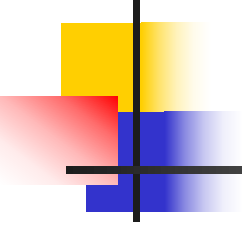


2.7 类型转换

- C语言允许双精度、单精度、整型及字符数据之间混合运算

$10 + '1' + 6.5$

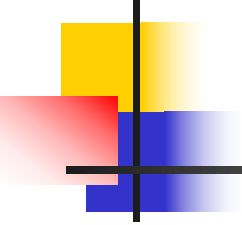
但有一个规则：先转换成同一类型,再计算。



整数提升

- 任何表达式中的**char**、**unsigned char**、**short**和**unsigned short**都要先转换成**int**或**unsigned**，如果原始类型的所有值可以用**int**表示，则转换成**int**，否则转换成**unsigned**，把这称为“整数提升”。
- **short a;**

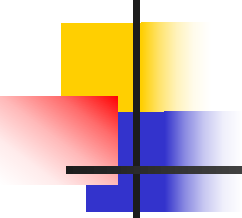
sizeof(a)=?	sizeof(-a)=?
2	4



算术转换

**char / short → int → unsigned → long → unsigned long
→ float → double → long double**





赋值转换

- 右操作数的值被转换为左操作数的类型
- 例如:

short s=5; double d=2.9; long l=0x11118001;
则

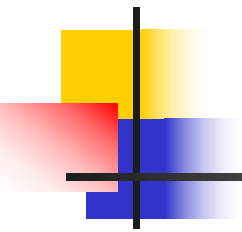
s = d; /*把d转换为short, 再赋给s。值为2*/

d = s; /*把s转换为double, 再赋给d。值为5.0*/

s = l; /* s值为 ? */

printf("%hx %hd %hu\n",s,s,s);

// 输出 8001 -32767 32769

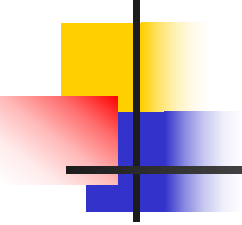


```
int n=(1<<31)-1;
```

```
float x=n;
```

```
printf("%d,%f\n",n,x);
```

- 结果一样吗



强制类型转换

■ (类型名) 操作数

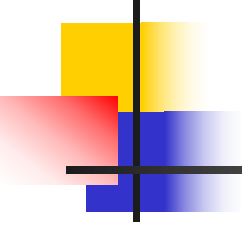
■ 例如,

`(double) i` /* 将i转换成double, i的类型保持不变*/

`(long)('a'-32)` /* 'a' 被自动转换成int,
相减的结果被强制转换为long*/

`(float)x+y` /*等价于((float)x)+y*/

`(double)x=10` /*错误*/



2.8 枚举类型

- 使代码更具可读性，理解清晰，易于维护。
- 它是由用户定义的若干枚举常量的集合
- 枚举常量用标识符命名,缺省状态下，其值是所有列举元素的序号，序号从**0**开始

```
enum color { WHITE,YELLOW,RED,BLUE};
```

```
enum week { SUN, MON, TUE, WED, THU, FRI, SAT };
```

枚举类型的定义

```
enum week { SUN, MON, TUE, WED, THU, FRI, SAT } ;
```

↑
关键字

↑
枚举名

枚举常量

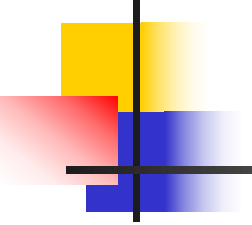
在未指定值的缺省情况下，第一个枚举常量的值为0，以后的值依次递增1。

可以指定一个或多个枚举常量的值，未指定值的枚举常量的值比前面的值大1。

```
enum sizes { SMALL, MEDIUM=10, BIG , TOO_BIG=20 } ;
```

enum 后面也可以不出现枚举名。

```
enum { WIN, LOSE, TIE, ERROR} ;
```

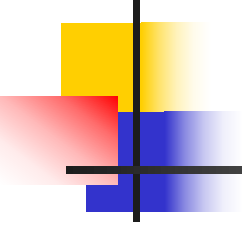



用枚举类型定义符号常量

```
#define WIN      0
#define LOSE     1
#define TIE      2
#define ERROR    -1
```

- 可用下面的枚举类型定义来代替。

```
enum { WIN, LOSE, TIE, ERROR=-1};
```



枚举变量的声明

(1) 在定义枚举类型的同时说明枚举变量

```
enum color { RED, GREEN, BLUE} c1, c2;
```

(2) 利用枚举名来说明枚举变量

```
enum color { RED, GREEN, BLUE} c1;
```

```
enum color c2;
```

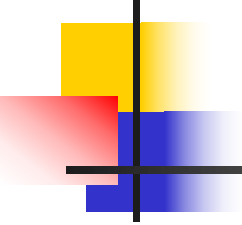
或者

```
enum color { RED, GREEN, BLUE};
```

```
enum color c1, c2;
```

枚举变量的输入和输出

- 一个枚举变量的值是int型整数，其值域仅限于列举的范围，枚举变量值的输入和输出都只能是整数。
- **c1=BLUE;** /* 等价于 **c1=2;** 使用枚举常量有助于读者理解程序 */
printf(“%d”,c1); /* 输出**2**，而不是**BLUE** */
scanf(“%d”,c2); /* 输入**0**，不能输入**RED** */
- 面的语句是错误的：
c1=3; /* 变量**c1**的值域为：**0、1和2** */
printf(“%s”, GREEN);
/* 输出错误的结果，而不是**GREEN** */
if(c1==RED) printf(“Red”);



输出对应的英文名

- 一个枚举变量的值是一个**int**整数，它可以出现在整数允许出现的任何地方。如果要输出与枚举值相对应的枚举常量标识符或代表其含义的完整字符串，可以定义一个**字符串数组**，以枚举值作为下标。
- ```
enum color { RED, GREEN, BLUE};
enum color c1;
char *colorName[]={ "Red", "Green", "Blue"};
c1 = BLUE;
printf("%s",colorName[c1]);
```
- 自学例2.30